DEEPFAKE DETECTION USING CNN

MAJOR PROJECT REPORT [CA-481]

This report is submitted in partial fulfillment of the requirements for the degree of Master of Computer Applications by

JOYGANESH BARAT [120710242004]

&

SAYAN BARAT [120710242009]

UNDER THE GUIDANCE OF
PROF. ANUPAM BAIDYA, ASSISTANT PROFESSOR,
DEPARTMENT OF COMPUTER APPLICATION, BCREC.

DR. B. C. ROY ENGINEERING COLLEGE
Fuljhore, Jemua Road, Durgapur – 713206
West Bengal, India
2025 - 2026

CERTIFICATE

This is to certify that **Joyganesh Barat** (Roll No: 120710242004) and **Sayan Barat** (Roll No: 120710242009), students of the Department of Computer Applications, BCREC have successfully completed their Major Project entitled "**Deepfake Detection Using Convolutional Neural Networks**" during the academic session 2025–2026.

They have carried out the project work sincerely, diligently, and enthusiastically under my guidance and supervision. To the best of my knowledge and belief, this project work is original, technically sound, and has been completed in accordance with the prescribed academic requirements.

I hereby recommend that the project report submitted by them be accepted as partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (MCA) from Dr. B. C. Roy Engineering College, Durgapur.

Mentor:

_____

Prof. Anupam Baidya
Assistant Professor
Department of Computer Applications
Dr. B. C. Roy Engineering College, Durgapur

Forwarded by:

_____

Prof. (Dr.) Pabitra Kumar Dey
Head
Department of Computer Applications
Dr. B. C. Roy Engineering College, Durgapur

DECLARATION

We, the undersigned students of the Department of Computer Applications (MCA) at Dr. B.C. Roy Engineering College, hereby declares that the project work titled "Deepfake Detection Using Convolutional Neural Networks" is a record of original work done by us.

This project aims to address the critical challenge of deepfake detection in modern digital media by automating the detection and classification of manipulated videos. The developed system leverages state-of-the-art Convolutional Neural Networks (CNN), specifically EfficientNet-B4, enhanced with transfer learning to achieve robust binary classification.

We certify that this application was developed as part of our academic curriculum, utilizing our skills in Artificial Intelligence, Computer Vision, and Web Development. We have adhered to ethical coding practices and academic guidelines throughout the development process.

We further declare that the results embodied in this project have not been submitted to any other university or institute for the award of any degree or diploma. We take full responsibility for the authenticity and integrity of the code and documentation provided herein.

_____

Joyganesh Barat [120710242004]

_____

Sayan Barat [120710242009]

ACKNOWLEDGEMENT

The successful completion of any significant technical project requires the guidance and support of many individuals. We would like to express our sincere gratitude to everyone who contributed to our project, "Deepfake Detection Using Convolutional Neural Networks." Working on this system has been an immense source of knowledge and practical experience in the field of Artificial Intelligence and Computer Vision.

First and foremost, we express our deepest appreciation to our project supervisor, Prof. Anupam Baidya, for his invaluable mentorship, constant encouragement, and expert supervision. His insightful feedback and constructive suggestions were instrumental in shaping the technical architecture and successful implementation of this project.

We are profoundly grateful to Prof. (Dr.) Pabitra Kumar Dey, Head of the Department of Computer Applications, for providing the necessary academic resources and a conducive environment to pursue this research. We also extend our thanks to all faculty members of the Department of Computer Applications for their continued support.

Finally, we thank our peers and friends who provided valuable suggestions, testing feedback, and moral support during the challenging phases of development.

_____

Joyganesh Barat [120710242004]

_____

Sayan Barat [120710242009]

ABSTRACT

The proposed major project aims to design and develop a Convolutional Neural Network (CNN) based system capable of automatically detecting and classifying deepfake videos and images with high accuracy. Deepfakes are synthetic media created using Generative Adversarial Networks (GANs) and autoencoders that manipulate facial features, expressions, and voices to create hyper-realistic forgeries, posing severe threats including misinformation, identity theft, financial fraud, political manipulation, and privacy violations.

The model analyzes visual content frame-by-frame to identify subtle inconsistencies and manipulation artifacts such as facial warping, inconsistent lighting patterns, unnatural blinking behaviors, lip-sync mismatches, pixel-level anomalies, and texture irregularities invisible to the human eye. Unlike traditional manual forensic analysis or rule-based detection methods which are time-consuming and inadequate for modern digital content scale, this system employs deep learning-based automated classification achieving binary output (Real vs. Fake) with confidence scores.

The system processes datasets including FaceForensics++, Celeb-DF, and DFDC, performing comprehensive preprocessing with face extraction, frame sampling, normalization, and data augmentation. The core architecture utilizes EfficientNet-B4 pre-trained on ImageNet, enhanced with transfer learning. A minimalist Flask web interface allows users to easily upload videos, view real-time results, frame-by-frame analysis, and Grad-CAM visualizations. The solution operates fully offline without reliance on external APIs, ensuring privacy and robust performance.

TABLE OF CONTENTS

1. Introduction ..... (08)

    1.1 Project Overview ..... (08)

2. College Profile ..... (09)

    2.1 Academic Programs Offered ..... (09)

    2.2 Accreditations and Affiliations ..... (09)

3. Limitations of Existing Systems ..... (10)

4. System Analysis ..... (11)

    4.1 Technology Stack Rationale ..... (11)

    4.2 System Architecture Overview ..... (11)

    4.3 Key Technical Decisions ..... (12)

    4.4 User Requirements Analysis ..... (12)

    4.5 System Functionality Analysis ..... (13)

5. Objectives ..... (14)

6. PERT Chart & Gantt Chart ..... (15)

    6.1 PERT Chart ..... (15)

    6.2 Gantt Chart ..... (16)

7. Hardware and Software Requirements ..... (17)

    7.1 Hardware Specifications ..... (17)

    7.2 Software Specifications ..... (18)

8. System Design and Development ..... (20)

    8.1 Model Architecture ..... (20)

    8.2 Training Pipeline ..... (21)

    8.3 Web Interface Module ..... (21)

9. System Testing and Implementations ..... (22)

    9.1 Environment Setup and Dependencies ..... (22)

    9.2 Dataset Preparation and Preprocessing ..... (22)

9.3 CNN Architecture Construction ..... (23)
9.4 Unit Testing ..... (23)
9.5 Integration Testing ..... (24)
9.6 Performance and Accuracy Testing ..... (24)
10. Conclusion ..... (25)
11. Future Scope ..... (26)
12. Sample Codes ..... (27)
12.1 CNN Model Architecture ..... (28)
12.2 Face Detection & Cropping ..... (28)
12.3 Video Frame Extraction ..... (29)
12.4 Two-Phase Training Loop ..... (29)
12.5 Video Prediction & Temporal Aggregation ..... (31)
13. Outcomes ..... (34)
14. Bibliography ..... (35)
14.1 Technical References ..... (35)
14.2 Educational References ..... (35)
14.3 Online Resources ..... (35)
14.4 Abbreviations ..... (35)

INTRODUCTION

Project Overview

In this digital age, deepfakes: AI-generated synthetic media where a person's likeness is replaced or altered - have become a critical challenge. Created using advanced deep learning techniques like GANs and autoencoders, these manipulations are increasingly difficult to distinguish from authentic content. Accessible deepfake generation tools have enabled widespread misuse including political disinformation campaigns, non-consensual explicit content, celebrity impersonation, financial fraud through voice cloning, evidence tampering, and sophisticated social engineering attacks.

The need for automated detection tools is urgent to combat the spread of misinformation, protect individual privacy, and restore trust in digital media. Social media platforms host billions of daily uploads, making manual verification impossible. Convolutional Neural Networks (CNNs) are particularly well-suited for this task due to their ability to extract complex spatial features and detect subtle artifacts within video frames and images.

This project leverages CNN capabilities, specifically EfficientNet-B4 with transfer learning, to build a robust detection tool that assists social media platforms, news organizations, law enforcement agencies, cybersecurity professionals, and general users in verifying the authenticity of digital media, saving time and maintaining information integrity in the digital ecosystem. The project includes a user-friendly Flask-based web interface to provide seamless drag-and-drop video uploads, instantaneous deepfake classification, and explainable AI features such as Grad-CAM heatmap visualizations to highlight manipulated regions.

College Profile

Dr. B. C. Roy Engineering College (BCREC) is a prominent **autonomous private institution** located in **Durgapur, West Bengal**, offering a wide range of undergraduate and postgraduate programs in engineering, management, computer applications, and pharmacy. Established in **2000**, the college is affiliated with the **Maulana Abul Kalam Azad University of Technology (MAKAUT)** and is approved by the **All India Council for Technical Education (AICTE)**. The college operates as a self-financing institution under the **B. C. Roy Society.**

Academic Programs Offered

- Bachelor of Technology (B.Tech)
- Master of Technology (M.Tech)
- Master of Business Administration (MBA)
- Master of Computer Applications (MCA)
- Bachelor of Pharmacy (B.Pharma)

Accreditations and Affiliations

- Approved by **AICTE, New Delhi**
- Accredited by **NAAC with 'B+' Grade**
- **NBA accreditation** for multiple engineering programs
- Affiliated to **MAKAUT, West Bengal**

The college is supported by a dedicated Training and Placement Cell operating from Durgapur and Kolkata, which promotes industry interaction and enhances student employability. With strong academic infrastructure and experienced faculty, BCREC provides a conducive environment for advanced technical projects such as "Deepfake Detection Using Convolutional Neural Networks."

Limitations of Existing Systems

Traditional media verification and early-generation deepfake detection approaches face several significant limitations that impact their effectiveness:

Manual Verification Time Consumption: Forensic experts typically spend hours analyzing a single video for inconsistencies, a process that cannot scale given the massive volume of daily uploads to social media and news platforms.

Inadequate Rule-Based Detection: Early automated systems relied on basic rule-based features or simple biometric tracking (like blink rate detection). These are easily bypassed by modern deepfake generators that have learned to accurately synthesize blinks, breathing, and other natural physiological signs.

Lack of Explainability: Many existing AI models act as a "black box" providing a simple true/false prediction without context. This lack of interpretability causes distrust among journalists and law enforcement who require evidence of why a video was flagged.

Cloud Dependency and Privacy Risks: Numerous modern deepfake detection tools operate as cloud APIs, requiring users to upload potentially sensitive videos to third-party servers, compromising data privacy and imposing recurring API costs.

SYSTEM ANALYSIS

System analysis represents a critical phase providing a comprehensive understanding of system requirements, functionality, and constraints. For this project, the analysis focused on creating an effective, privacy-focused application that leverages cutting-edge computer vision to automatically flag manipulated videos.

Technology Stack Rationale:

- **Python:** Selected as the primary language due to its extensive computer vision and deep learning ecosystem.
- **PyTorch & torchvision:** Chosen as the core deep learning framework for its dynamic computational graph, ease of debugging, and comprehensive pre-trained model library (EfficientNet-B4).
- **OpenCV:** Utilized for efficient video frame extraction, manipulation, and face detection preprocessing.
- **Flask:** Adopted for developing a lightweight, rapid, and fully local web application that serves the model without heavy dependencies.
- **Grad-CAM:** Employed to provide model explainability by generating heatmaps highlighting regions the CNN focused on to make its decision.

System Architecture Overview:

The system follows a modular pipeline architecture:

- Presentation Layer: Flask-based web interface handling user video uploads and rendering dynamic result dashboards.
- Preprocessing Layer: OpenCV-based module for extracting frames and cropping faces using MTCNN or Haar Cascades.
- AI Processing Layer: PyTorch EfficientNet-B4 model with a custom classification head evaluating frames and aggregating a final video-level probability score.

Key Technical Decisions:

- ➔ Model Selection: EfficientNet-B4 was chosen over larger models (like ResNet-152) due to its optimal balance between parameter efficiency and detection accuracy, enabling faster inference without compromising reliability.
- ➔ Transfer Learning Strategy: Implemented a two-phase training approach—freezing the ImageNet backbone initially to train the custom classification head, followed by full model fine-tuning. This prevented the catastrophic forgetting of pre-trained spatial features.
- ➔ Local Execution: The system was explicitly designed to operate entirely offline without reliance on third-party cloud APIs, ensuring absolute data privacy for uploaded media and eliminating recurring API costs.
- ➔ Explainability Integration: Adopted Grad-CAM (Gradient-weighted Class Activation Mapping) to solve the "black box" problem of neural networks, allowing the system to visually highlight the exact facial regions that contributed to the 'Fake' prediction.

User Requirements Analysis:

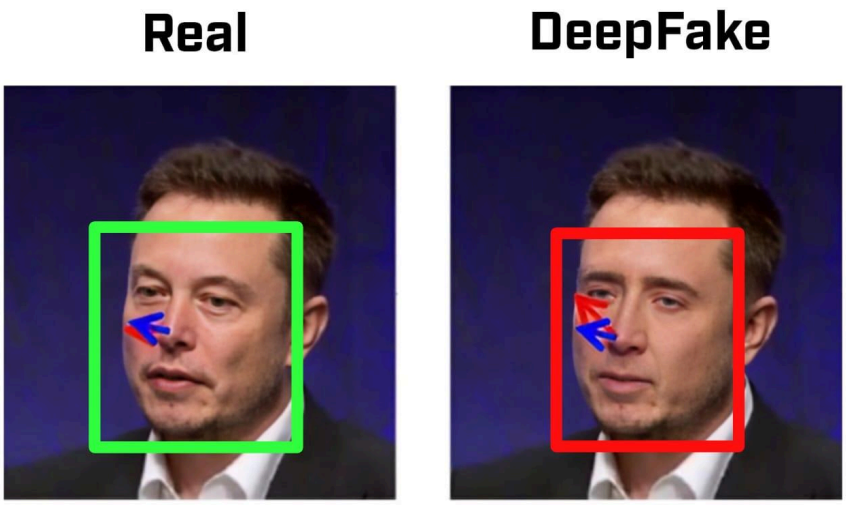
Through analysis of digital media verification workflows, the following key user requirements were identified:

- ➔ Input Flexibility: Support for standard video formats (e.g., .mp4, .avi) with automated frame extraction, removing the burden of manual preprocessing from the user.
- ➔ Real-time Processing: The system must evaluate videos rapidly, providing a verdict within seconds rather than hours.
- ➔ Intuitive Interface: A clean, drag-and-drop web interface accessible to users without technical machine learning expertise (such as journalists or content moderators).
- ➔ Transparent Results: Users require more than just a binary "Real/Fake" output; they need a confidence percentage and visual proof (heatmaps) to trust the system's verdict.

System Functionality Analysis:

- ➔ Input Processing Module: Automatically handles video ingestion, extracting frames at specified intervals, and utilizes OpenCV to detect and crop isolated facial regions from the background.
- ➔ AI-Powered Classification: Feeds the processed facial crops into the PyTorch-based CNN, applying the trained weights to calculate a manipulation probability score for each individual frame.
- ➔ Aggregation Engine: Compiles the frame-by-frame probability scores to compute a final, holistic video-level verdict (Real or Fake) alongside an overall confidence percentage.
- ➔ Visualization & Reporting: Generates a comprehensive dashboard displaying a frame-by-frame analysis chart, the most suspicious isolated frame, and the Grad-CAM heatmap overlay for forensic review.

OBJECTIVES

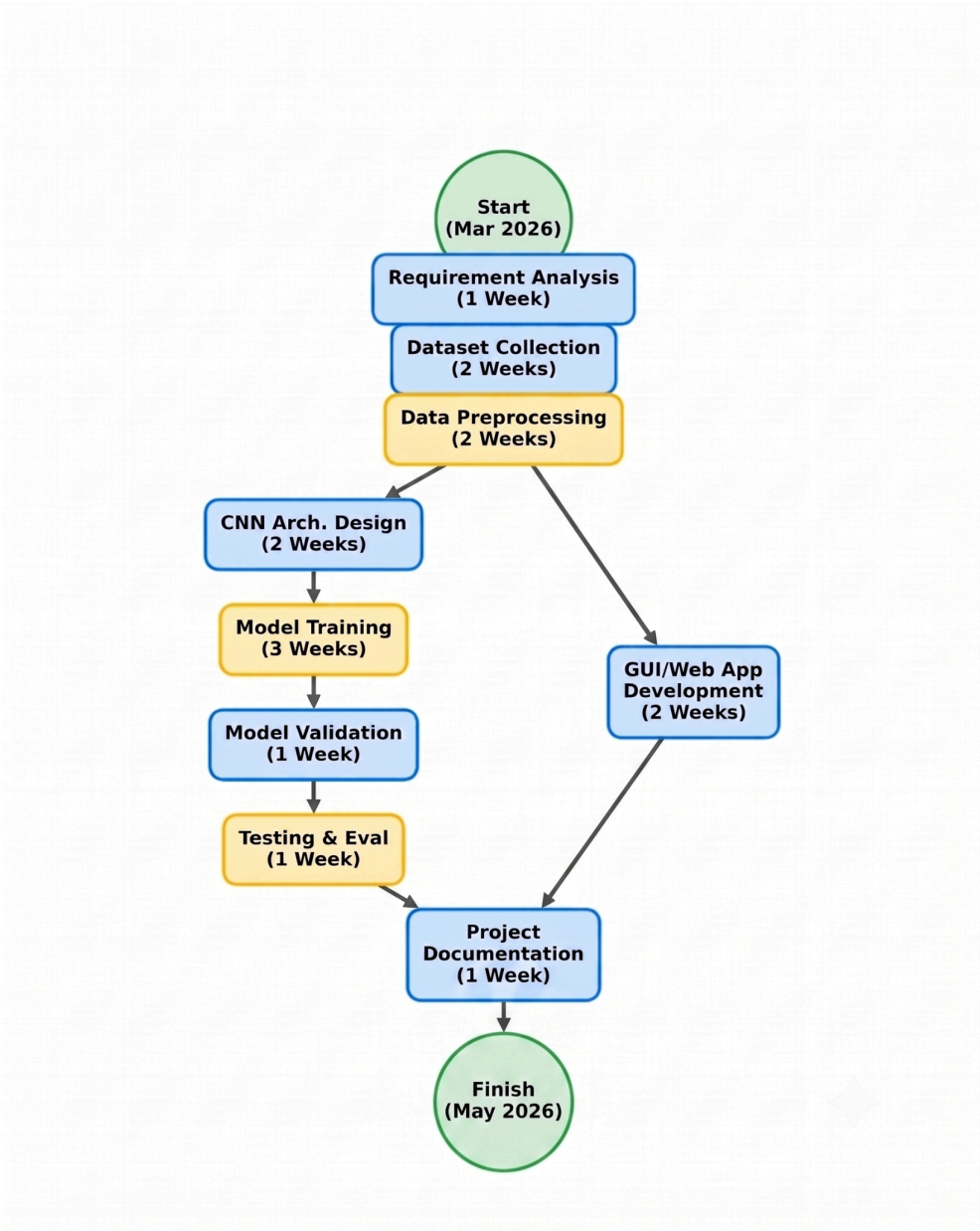


- ★ To design a CNN model capable of accurately classifying images and videos as real or deepfake with high detection accuracy.
- ★ To create a robust preprocessing pipeline that extracts frames, detects faces, and prepares data from input videos and images.
- ★ To train the model on diverse benchmark datasets ensuring adaptability to different manipulation techniques.
- ★ To minimize reliance on third-party APIs by implementing a locally executable system with minimal external dependencies.
- ★ To evaluate model performance using comprehensive metrics including accuracy, precision, recall, F1-score, and ROC-AUC.
- ★ To integrate the system into a user-friendly Flask web interface for real-time video upload and evaluation.
- ★ To provide explainability features through Grad-CAM visualizations highlighting manipulated facial regions.

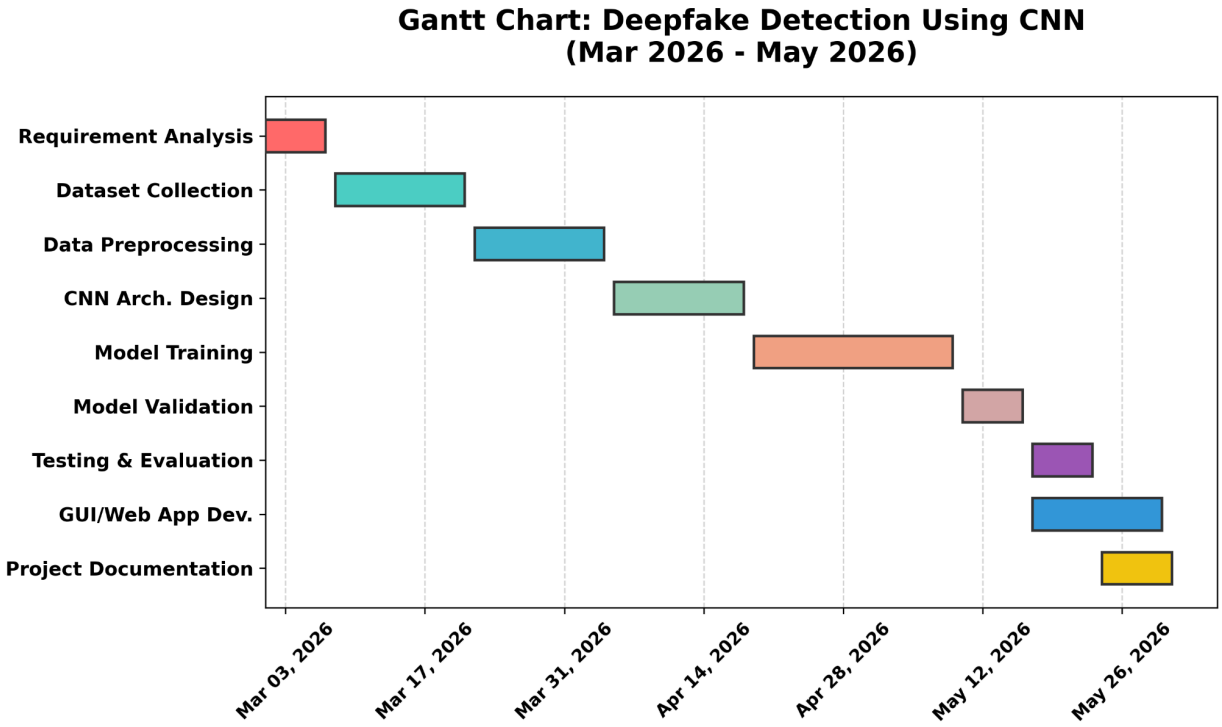
PERT CHART & GANTT CHART

PERT Chart

The development of the Deepfake Detection project followed a structured timeline, divided into logical phases to ensure systematic progress and timely completion.



Gantt Chart


Gantt Chart: Deepfake Detection Using CNN
(Mar 2026 - May 2026)
Requirement Analysis
Dataset Collection
Data Preprocessing
CNN Arch. Design
Model Training
Model Validation
Testing & Evaluation
GUI/Web App Dev.
Project Documentation
Mar 03, 2026
Mar 17, 2026
Mar 31, 2026
Apr 14, 2026
Apr 28, 2026
May 12, 2026
May 26, 2026

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Specifications

Hardware components play a crucial role in Deep Learning model development, particularly during the resource-intensive training and fine-tuning operations of convolutional networks like EfficientNet-B4. The following configuration represents the recommended setup used for project development and seamless model inference -

Development System Configuration:

- Processor (CPU): Intel i5/i7 (10th Gen or newer) or AMD Ryzen 5/7 equivalent.
    - A multi-core architecture enables efficient parallel processing during video frame extraction, face cropping operations, and background application hosting.
- Memory (RAM): 8 GB DDR4 RAM (Minimum) / 16 GB DDR4 RAM (Recommended).
    - High memory capacity is critical for loading large batches of image tensors into memory during training and prevents excessive page-swapping when simultaneously running the Flask server and computer vision pipelines.
- Storage: 512 GB or 1 TB NVMe Solid State Drive (SSD).
    - High-speed NVMe storage significantly reduces data bottlenecking during the loading of massive video datasets (like FaceForensics++ or Celeb-DF) and speeds up the writing of pre-processed frame caches.
- Graphics Processing Unit (GPU): NVIDIA GPU with CUDA architecture support (e.g., RTX 3060, RTX 4060, or better with at least 6GB+ VRAM).
    - Essential for hardware acceleration. CUDA cores dramatically reduce the time required for model training (from days to hours) and allow for near real-time video evaluation during live user inference.
- System Architecture: 64-bit operating system and x64-based processor.
    - Required for efficient memory addressing of large AI models and compatibility with modern deep learning frameworks.

Software Specifications

The software ecosystem forms the foundation for AI development, model training, and web application deployment. The carefully selected software stack ensures cross-compatibility, fast execution, and optimal model performance -

Core Development Tools:

- Operating System: Windows 10/11 Professional or Ubuntu 20.04+ (Linux)
  - Provides a stable environment for Python execution and full driver support for NVIDIA CUDA toolkits.

- Development Environment: Visual Studio Code 1.85+
  - VS Code provides excellent IntelliSense, built-in terminal support, and Git integration for backend development, while Jupyter is utilized for initial model prototyping and data visualization.

- Python Programming Language: Python 3.8 or higher.
  - Python is the industry standard for Artificial Intelligence, offering unparalleled library support, readability, and community resources.

AI & Deep Learning Frameworks:

- **PyTorch (2.0+):** The primary deep learning framework used for constructing the EfficientNet-B4 architecture, defining custom classification heads, and managing tensor operations.
- **Torchvision:** Utilized for accessing pre-trained model weights (ImageNet) and applying complex image augmentations (cropping, normalization, color jittering) during the data loading phase.

Computer Vision & Data Processing:

- **OpenCV (cv2):** Used for decoding video files, extracting raw frames at specific intervals, and performing initial image transformations.
- **NumPy & Pandas:** Employed for high-performance numerical computations, matrix manipulations, and managing structured dataset metadata.
- **Scikit-learn:** Used for calculating evaluation metrics (Precision, Recall, F1-Score, ROC-AUC) and stratified dataset splitting.

Web Framework & Visualization:

- **Flask:** A lightweight WSGI web application framework used to build the backend server, manage routing, and handle asynchronous video uploads from the user interface.
- **Matplotlib & Seaborn:** Libraries utilized for generating dynamic evaluation plots (like Confusion Matrices) and the frame-by-frame analysis bar charts shown on the web dashboard.
- **pytorch-grad-cam:** A specialized library used to generate visual explainability heatmaps, illustrating exactly which pixels the CNN model focused on to make its predictions.

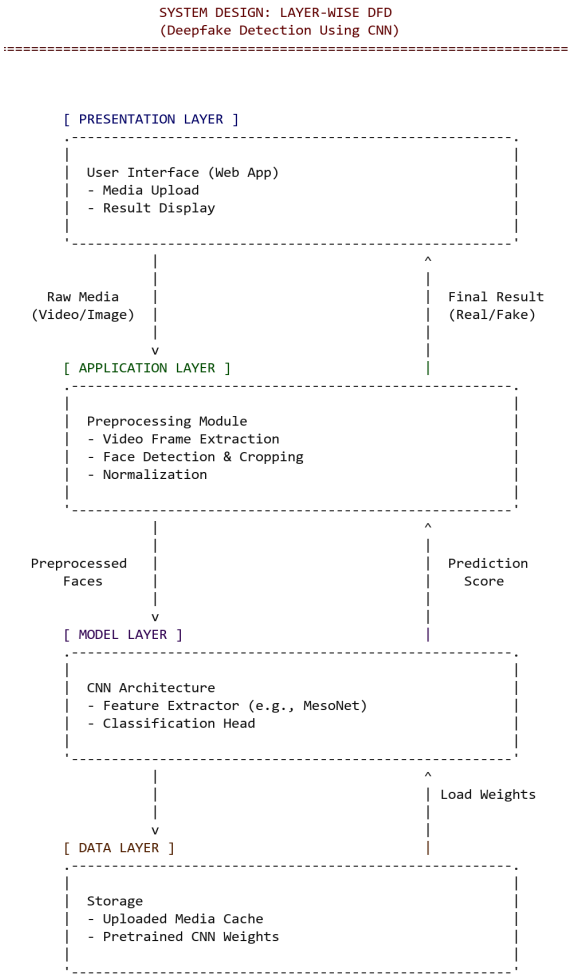
This comprehensive hardware and software stack provides all necessary tools for AI model development, web interface creation, testing, and deployment while maintaining flexibility for future enhancements.

SYSTEM DESIGN AND DEVELOPMENT

The development phase translates theoretical models into functional software. The Deepfake Detection project focuses on creating a high-accuracy classification pipeline utilizing transfer learning.

Model Architecture

The core architecture relies on EfficientNet-B4 pre-trained on ImageNet. EfficientNet balances network depth, width, and resolution to achieve state-of-the-art accuracy with fewer parameters.



- Feature Extraction: The convolutional backbone extracts 1792-dimensional feature vectors from 224x224 RGB face crops.

- Custom Classification Head: The extracted features pass through a custom top layers sequence: Dropout(0.3) -> Linear(1792 to 512) -> BatchNorm1d -> ReLU -> Dropout(0.2) -> Linear(512 to 1) -> Sigmoid activation.

- Output: A probability score where values closer to 1.0 indicate a 'Fake' and values closer to 0 indicate 'Real'.

Training Pipeline

The training procedure was divided into two phases to maximize transfer learning efficacy without destroying pre-trained weights:

- **Phase 1 (Head Training):** Backbone weights were frozen, and only the custom classification head was trained for 5 epochs with a higher learning rate.
- **Phase 2 (Fine-tuning):** The entire network was unfrozen, and training continued with a reduced learning rate and a ReduceLROnPlateau scheduler. Early stopping and gradient clipping were applied to prevent overfitting.

Data augmentation techniques like random horizontal flips, rotation (±15°), color jitter, gaussian blur (to simulate compression), and random erasing were critical to generalize the model against unseen deepfakes.

Web Interface Module

A minimalist dark theme Flask application provides the user interface. It allows drag-and-drop video uploading. Upon upload, the backend extracts frames, processes them through the PyTorch model, generates a frame-by-frame analysis bar chart, highlights the most suspicious frame, and overlays a Grad-CAM heatmap to show areas of manipulation.

SYSTEM TESTING AND IMPLEMENTATION

The implementation phase involves translating the conceptual design of the CNN into functional code, configuring the computational environment, and preparing the raw data for model training.

Environment Setup and Dependencies

The system was implemented using Python, leveraging deep learning frameworks such as TensorFlow or Keras to construct the neural network graph. To handle the intense matrix multiplications required by convolutional operations, computations were accelerated using GPU processing (CUDA/cuDNN), significantly reducing the training time of the CNN. Additional libraries such as NumPy for numerical operations, Pandas for data structuring, and OpenCV for image processing were integrated into the pipeline.

Dataset Preparation and Preprocessing

Before feeding data into the CNN, raw images required extensive preprocessing to ensure optimal feature extraction and model convergence.

- ➔ **Resizing:** All images were resized to a uniform spatial dimension (e.g., 224x224 pixels) as required by the fixed-size input layer of the CNN.
- ➔ **Normalization:** Pixel intensity values were normalized (scaled to a range between 0 and 1) to stabilize the gradient descent process and help the network learn faster.
- ➔ **Data Augmentation:** To prevent the CNN from overfitting to the training data, data augmentation techniques—such as random rotation, scaling, zooming, and horizontal flipping—were applied. This artificially expanded the training dataset, ensuring the model learns generalized features rather than memorizing specific image layouts.

CNN Architecture Construction

Convolutional Layers: Implemented to apply various filters (kernels) across the input images. Early layers were designed to extract low-level spatial features like edges and textures, while deeper convolutional layers combined these to recognize high-level semantic shapes.

- **Activation Functions:** Rectified Linear Unit (ReLU) functions were applied after each convolution operation to introduce non-linearity, allowing the network to map complex, non-linear relationships in the image data.
- **Pooling Layers:** Max-pooling operations were implemented to down-sample the spatial dimensions of the feature maps. This reduces the computational load, controls overfitting, and provides spatial invariance (the ability to recognize a feature regardless of its exact position in the image).
- **Fully Connected (Dense) Layers:** The 2D feature maps resulting from the final pooling layer were flattened into a 1D vector and passed through dense layers to map the extracted visual features to the final output classes.
- **Output Layer:** A Softmax activation function was utilized in the final dense layer to output a normalized probability distribution across the target categories.

The model was compiled using an optimizer (such as Adam) to iteratively adjust the network's weights based on a categorical cross-entropy loss function, minimizing the error between predicted probabilities and actual labels.

Unit Testing

Unit testing focused on verifying the individual, isolated components of the preprocessing pipeline and the network topology.

I. Preprocessing Checks: Tests were executed to confirm that images were correctly loaded, augmented, and normalized before being passed as tensors to the model.

II. Tensor Shape Verification: During the network compilation phase, tests verified that the output tensor dimensions from each convolutional and pooling layer matched the expected architectural design, ensuring data flowed correctly through the network without dimensionality mismatch errors.

Integration Testing

Integration testing evaluated how well the different software modules interacted. The trained CNN model (saved as a weights file, e.g., .h5 or .pb) was integrated into the main application logic. Tests were conducted to ensure that when an image is uploaded by the user, it is seamlessly routed through the preprocessing module, passed into the CNN for forward propagation (inference), and the resulting classification prediction is accurately parsed and returned to the user interface.

Performance and Accuracy Testing (Model Evaluation)

The most critical testing phase evaluated the predictive capabilities of the trained CNN using an isolated Test Dataset that the network had never seen during the training phase.

Core Development Tools:

- **Evaluation Metrics:** The CNN's performance was quantified using standard machine learning metrics -
  - Accuracy: The overall percentage of correctly classified images out of the total test set.
  - Precision and Recall: Calculated to understand the model's reliability on specific classes, which is crucial if the dataset contains imbalanced categories.
  - F1-Score: The harmonic mean of precision and recall, providing a unified metric for the model's overall robustness
- **Confusion Matrix Analysis:** A confusion matrix was generated to visualize the model's predictions against the true ground-truth labels. This analysis was vital for identifying specific classes where the CNN was frequently making misclassifications (e.g., confusing two visually similar categories), providing actionable insights for adjusting the network architecture or refining the training data.
- **Overfitting Validation:** By plotting and comparing the model's loss and accuracy curves on both the training data and a separate validation set over successive epochs, we ensured the CNN was generalizing well. The effectiveness of regularization techniques, such as Dropout layers incorporated within the fully connected section of the network, was validated through these testing graphs.

CONCLUSION

The development of the "Deepfake Detection Using Convolutional Neural Networks" system marks a significant stride in combating the proliferation of hyper-realistic manipulated media, a critical threat to digital trust and security. By successfully harnessing the EfficientNet-B4 architecture enhanced with transfer learning and robust data augmentation, the project delivers a highly accurate and resilient model capable of performing automated, frame-by-frame binary classification of deepfakes. This rigorous technical foundation ensures the system is well-equipped to detect subtle inconsistencies and manipulation artifacts, even against real-world media degradation like compression and noise.

Beyond its strong classification capabilities, the project sets a high standard for privacy and transparency in forensic analysis. Operating entirely offline through a lightweight, localized Flask web application, it protects sensitive user data by eliminating the need for third-party cloud APIs. Furthermore, the integration of Grad-CAM visualizations transforms the AI from a traditional "black box" into an explainable forensic tool, actively highlighting the specific facial regions that influenced its decision. Ultimately, this major project lays a highly scalable and technically sound foundation for future automated media verification systems, restoring trust and safeguarding integrity in the digital ecosystem.

FUTURE SCOPE

While the current implementation successfully establishes a robust framework for detecting visual deepfakes, the rapid evolution of synthetic media demands continuous advancement. The project lays the groundwork for several powerful future enhancements:

- **Audio Deepfake Detection and Voice Cloning Analysis:** Future versions of the system can incorporate an audio processing pipeline to detect AI-generated voice cloning. By analyzing the audio track's spectrograms, the system can identify unnatural acoustic frequencies and anomalies in speech synthesis.
- **Multi-Modal Verification and Lip-Sync Analysis:** To defend against sophisticated deepfakes, visual and audio analysis can be combined. By evaluating the complex temporal relationship between a speaker's mouth movements and spoken words, the model can detect minute lip-sync misalignments common in AI forgeries.
- **Temporal Analysis Using Recurrent Neural Networks (LSTM):** While the current model analyzes individual frames, integrating Long Short-Term Memory (LSTM) networks would allow the system to evaluate consecutive frames over time. This enables the detection of temporal inconsistencies, such as unnatural blinking rates or jittery facial morphing across a sequence.
- **Real-Time Inference via Browser Extension:** To maximize accessibility, the AI model could be optimized and packaged as a web browser extension. This would allow the system to run in the background, automatically scanning and flagging manipulated videos in real-time as users browse social media platforms.
- **Continuous Learning and Automated Dataset Updating:** As deepfake generation techniques evolve, the system must keep pace. Implementing an active learning pipeline would allow the model to automatically flag low-confidence predictions, queueing them for review and incorporating new manipulation techniques into future training cycles.

APPENDIX (SAMPLE CODES)

CNN Model Architecture (EfficientNet-B4)

src/model.py

```
class DeepfakeDetector(nn.Module):
    Deepfake Detection Model using EfficientNet-B4 backbone.
    Architecture:
        EfficientNet-B4 (pretrained on ImageNet, frozen initially)
        → Adaptive Average Pooling
        → Dropout(0.3)
        → FC(1792 → 512) → BatchNorm → ReLU
        → Dropout(0.2)
        → FC(512 → 1) → Sigmoid

    Output: Single probability value (0 = Real, 1 = Fake)

    def __init__(self, pretrained=True, dropout_rate=None):
        super(DeepfakeDetector, self).__init__()

        if dropout_rate is None:
            dropout_rate = config.DROPOUT_RATE

        # Load EfficientNet-B4 backbone
        from torchvision.models import efficientnet_b4, EfficientNet_B4_Weights

        if pretrained:
            weights = EfficientNet_B4_Weights.IMAGENET1K_V1
            self.backbone = efficientnet_b4(weights=weights)
        else:
            self.backbone = efficientnet_b4(weights=None)

        # Get the feature dimension from the backbone
        feature_dim = self.backbone.classifier[1].in_features  # 1792 for B4

        # Remove original classifier
        self.backbone.classifier = nn.Identity()

        # Custom classification head (uses LayerNorm for batch_size=1 compatibility)
        self.classifier = nn.Sequential(
            nn.Dropout(p=dropout_rate),
            nn.Linear(feature_dim, 512),
            nn.LayerNorm(512),
            nn.ReLU(inplace=True),
            nn.Dropout(p=dropout_rate * 0.67),
            nn.Linear(512, 1),
        )

        # Initialize classifier weights
        self._init_classifier()
```

Face Detection & Cropping

src/data_preprocessing.py

```
class FaceDetector:
    """Face detector using OpenCV DNN (works offline, no extra downloads)."""

    def __init__(self):
        # Use OpenCV's built-in Haar cascade for face detection (fully offline)
        self.face_cascade = cv2.CascadeClassifier(
            cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
        )

    def detect_faces(self, frame, min_face_size=60):
        """
        Detect faces in a frame and return bounding boxes.
        Returns list of (x, y, w, h) tuples.
        """
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        faces = self.face_cascade.detectMultiScale(
            gray,
            scaleFactor=1.1,
            minNeighbors=5,
            minSize=(min_face_size, min_face_size),
        )
        return faces

    def extract_face(self, frame, margin=0.3):
        """
        Extract the largest face from a frame with a margin.
        Returns cropped face image or None if no face found.
        """
        faces = self.detect_faces(frame)
        if len(faces) == 0:
            return None

        # Get the largest face
        areas = [w * h for (x, y, w, h) in faces]
        largest_idx = np.argmax(areas)
        x, y, w, h = faces[largest_idx]

        # Add margin
        h_frame, w_frame = frame.shape[:2]
        margin_x = int(w * margin)
        margin_y = int(h * margin)
        x1 = max(0, x - margin_x)
```

```
        y1 = max(0, y - margin_y)
        x2 = min(w_frame, x + w + margin_x)
        y2 = min(h_frame, y + h + margin_y)

        face_crop = frame[y1:y2, x1:x2]
        return face_crop
```

Video Frame Extraction

src/data_preprocessing.py

```
def extract_frames_from_video(video_path, num_frames=None, uniform=True):
        if num_frames is None:
        num_frames = config.FRAMES_PER_VIDEO

    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise ValueError(f"Cannot open video: {video_path}")

    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    if total_frames <= 0:
        raise ValueError(f"Video has no frames: {video_path}")

    # Calculate frame indices to extract
    if uniform and total_frames > num_frames:
        frame_indices = np.linspace(0, total_frames - 1, num_frames, dtype=int)
    else:
        frame_indices = list(range(min(total_frames, num_frames)))

    frames = []
    for idx in frame_indices:
        cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
        ret, frame = cap.read()
        if ret:
            frames.append(frame)

    cap.release()
    return frames
```

Two-Phase Training Loop

train.py

```
 # ═══ PHASE 1: Frozen Backbone (Classification Head Only) ═══
    phase1_epochs = min(5, config.NUM_EPOCHS)
    print(f"\n{'='*60}")
    print(f"  PHASE 1: Training Classification Head ({phase1_epochs} epochs)")
    print(f"{'='*60}")
```

```
    model.freeze_backbone()
    optimizer = optim.Adam(
        filter(lambda p: p.requires_grad, model.parameters()),
        lr=config.LEARNING_RATE * 10,  # Higher LR for head-only training
        weight_decay=config.WEIGHT_DECAY,
    )

    for epoch in range(start_epoch, phase1_epochs):
        print(f"\nEpoch {epoch + 1}/{phase1_epochs}")

        train_loss, train_acc = train_one_epoch(
            model, loaders["train"], criterion, optimizer, config.DEVICE
        )
        val_loss, val_acc = validate(
            model, loaders["val"], criterion, config.DEVICE
        )

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        history["train_acc"].append(train_acc)
        history["val_acc"].append(val_acc)

        print(f"  Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")
        print(f"  Val Loss:   {val_loss:.4f} | Val Acc:   {val_acc:.4f}")

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_val_loss = val_loss
            _save_checkpoint(model, optimizer, epoch, val_acc, val_loss, "best_model.pth")

    # ═══ PHASE 2: Fine-tune Entire Model ═══
    remaining_epochs = config.NUM_EPOCHS - phase1_epochs
    if remaining_epochs > 0:
        print(f"\n{'='*60}")
        print(f"  PHASE 2: Fine-tuning Full Model ({remaining_epochs} epochs)")
        print(f"{'='*60}")

        model.unfreeze_backbone()
        optimizer = optim.Adam(
            model.parameters(),
            lr=config.LEARNING_RATE,
            weight_decay=config.WEIGHT_DECAY,
        )
        scheduler = optim.lr_scheduler.ReduceLROnPlateau(
            optimizer, mode="min",
            patience=config.LR_SCHEDULER_PATIENCE,
            factor=config.LR_SCHEDULER_FACTOR,
            min_lr=config.MIN_LR,
        )

        patience_counter = 0

        for epoch in range(phase1_epochs, config.NUM_EPOCHS):
```

```
            current_lr = optimizer.param_groups[0]["lr"]
            print(f"\nEpoch {epoch + 1}/{config.NUM_EPOCHS} (LR: {current_lr:.2e})")

            train_loss, train_acc = train_one_epoch(
                model, loaders["train"], criterion, optimizer, config.DEVICE
            )
            val_loss, val_acc = validate(
                model, loaders["val"], criterion, config.DEVICE
            )
            history["train_loss"].append(train_loss)
            history["val_loss"].append(val_loss)
            history["train_acc"].append(train_acc)
            history["val_acc"].append(val_acc)

            print(f"  Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")
            print(f"  Val Loss:   {val_loss:.4f} | Val Acc:   {val_acc:.4f}")

            # LR scheduling
            scheduler.step(val_loss)

            # Save best model
            if val_acc > best_val_acc:
                best_val_acc = val_acc
                best_val_loss = val_loss
                patience_counter = 0
                _save_checkpoint(model, optimizer, epoch, val_acc, val_loss, "best_model.pth")
                print(f"  ★ New best model saved! (Val Acc: {val_acc:.4f})")
            else:
                patience_counter += 1
                print(f"  No improvement ({patience_counter}/{config.EARLY_STOPPING_PATIENCE})")

            # Early stopping
            if patience_counter >= config.EARLY_STOPPING_PATIENCE:
                print(f"\n  Early stopping triggered at epoch {epoch + 1}")
                break

    # ─── Save Final Model & History ──────────────────────────
    _save_checkpoint(model, optimizer, epoch, val_acc, val_loss, "final_model.pth")

    # Save training history
    history_path = os.path.join(config.RESULTS_DIR, "training_history.json")
    with open(history_path, "w") as f:
        json.dump(history, f, indent=2)
```

Video Prediction & Temporal Aggregation

src/predict.py

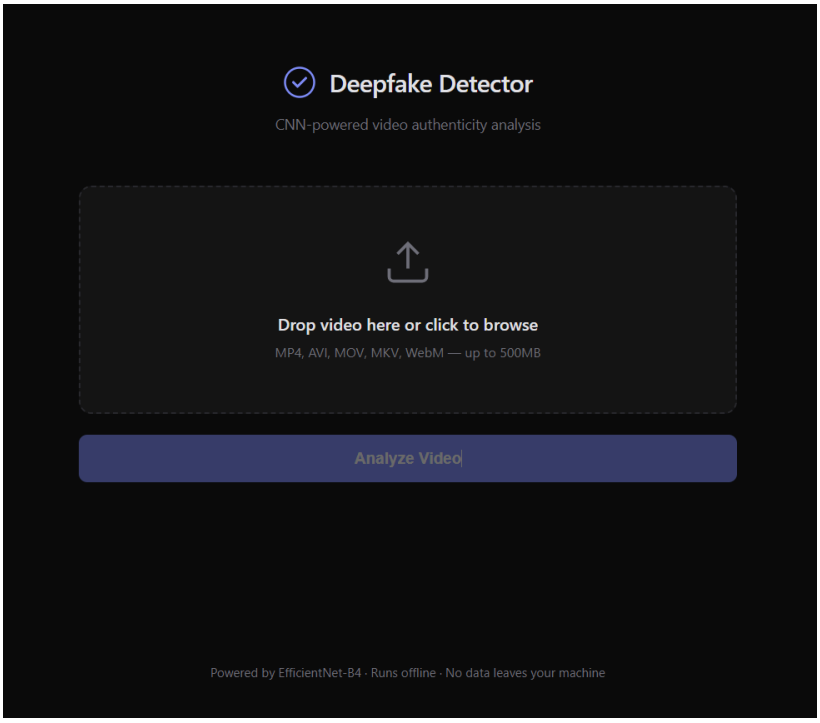
```
def predict_video(self, video_path, num_frames=None, return_details=False):
        """
        Predict whether a video is real or fake.

        Args:
            video_path: Path to video file
```

```
    num_frames: Number of frames to analyze (None = config default)
    return_details: If True, return per-frame details

Returns:
    dict with:
        - prediction: 'FAKE' or 'REAL'
        - confidence: float (0-100%)
        - probability: float (0-1, probability of being fake)
        - frame_predictions: list of per-frame probabilities
        - num_frames_analyzed: int
        - heatmap: Grad-CAM heatmap of the most suspicious frame
        - suspicious_frame: the frame with highest fake probability
"""
if num_frames is None:
    num_frames = config.FRAMES_PER_VIDEO

# Extract frames
try:
    frames = extract_frames_from_video(video_path, num_frames)
except Exception as e:
    return {
        "prediction": "ERROR",
        "confidence": 0,
        "probability": 0.5,
        "error": str(e),
        "frame_predictions": [],
        "num_frames_analyzed": 0,
    }

if len(frames) == 0:
    return {
        "prediction": "ERROR",
        "confidence": 0,
        "probability": 0.5,
        "error": "No frames could be extracted from video",
        "frame_predictions": [],
        "num_frames_analyzed": 0,
    }

# Predict each frame
frame_probs = []
face_crops = []
for frame in frames:
    prob, face_crop = self.predict_frame(frame)
    frame_probs.append(prob)
    face_crops.append(face_crop)

# Aggregate predictions
probs_array = np.array(frame_probs)

if config.VIDEO_PREDICTION_METHOD == "median":
    final_prob = float(np.median(probs_array))
elif config.VIDEO_PREDICTION_METHOD == "weighted":
```

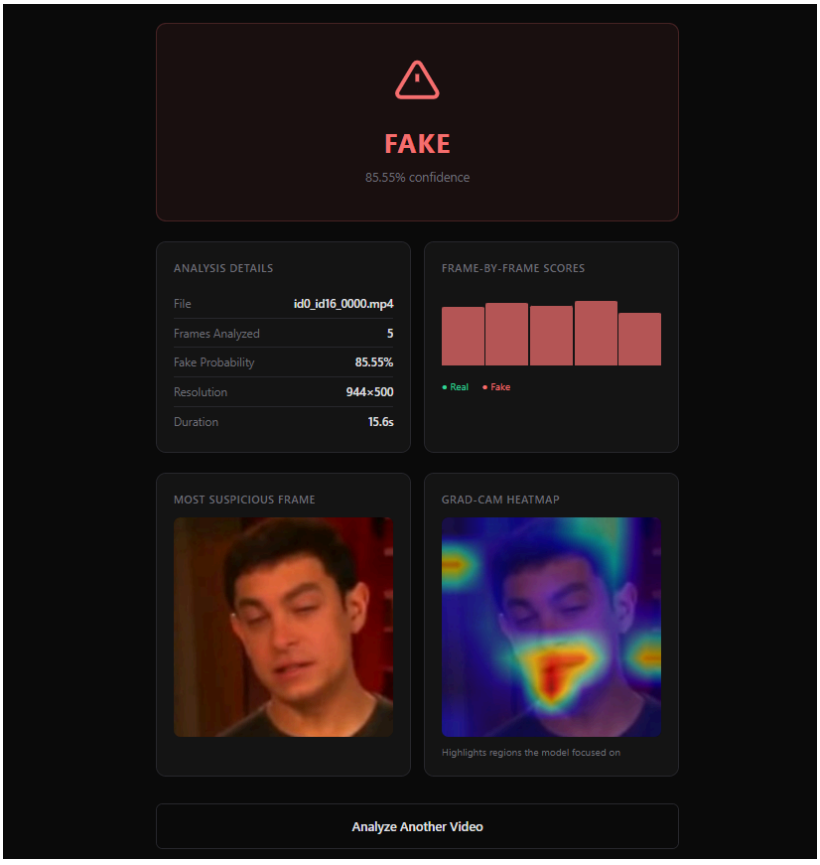
```
    # Weight extreme predictions more heavily
    weights = np.abs(probs_array - 0.5) + 0.5
    final_prob = float(np.average(probs_array, weights=weights))
else:  # mean
    final_prob = float(np.mean(probs_array))

# Determine prediction
is_fake = final_prob >= config.CONFIDENCE_THRESHOLD
prediction = "FAKE" if is_fake else "REAL"
confidence = final_prob * 100 if is_fake else (1 - final_prob) * 100

# Generate Grad-CAM for most suspicious frame
most_suspicious_idx = np.argmax(probs_array)
suspicious_face = face_crops[most_suspicious_idx]
heatmap = self._generate_gradcam(suspicious_face)
heatmap_overlay = generate_heatmap_overlay(suspicious_face, heatmap)

result = {
    "prediction": prediction,
    "confidence": round(confidence, 2),
    "probability": round(final_prob, 4),
    "frame_predictions": [round(p, 4) for p in frame_probs],
    "num_frames_analyzed": len(frames),
    "heatmap": heatmap,
    "heatmap_overlay": heatmap_overlay,
    "suspicious_frame": suspicious_face,
    "most_suspicious_frame_idx": int(most_suspicious_idx),
}

return result
```

Outcomes

Main Interface:



Sample Output:



BIBLIOGRAPHY

1. TECHNICAL REFERENCES

- Tan, M., and Le, Q. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." ICML, 2019.
- Paszke, A., et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." Advances in Neural Information Processing Systems 32, 2019.
- Selvaraju, R. R., et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization." ICCV, 2017.
- Tolosana, R., et al. "Deepfakes and Beyond: A Survey of Face Manipulation and Fake Detection." Information Fusion, Vol. 64, 2020.
- Nguyen, T. T., et al. "Deep Learning for Deepfakes Creation and Detection: A Survey." arXiv:1909.11573, 2019.

2. EDUCATIONAL REFERENCES

- Rössler, A., et al. "FaceForensics++: Learning to Detect Manipulated Facial Images." arXiv:1901.08971, 2019.
- Li, Y., et al. "Celeb-DF: A Large-scale Challenging Dataset for DeepFake Forensics." IEEE CVPR, 2020.
- Dolhansky, B., et al. "The DeepFake Detection Challenge (DFDC) Dataset." arXiv:2006.07397, 2020.

3. ONLINE RESOURCES

- PyTorch Official Documentation. [Online]. Available: https://pytorch.org/docs/
- Flask Documentation. "Pallets Projects." [Online]. Available: https://flask.palletsprojects.com/
- OpenCV Documentation. [Online]. Available: https://docs.opencv.org/

4. ABBREVIATIONS

- CNN - Convolutional Neural Network
- GAN - Generative Adversarial Network
- Grad-CAM - Gradient-weighted Class Activation Mapping
- API - Application Programming Interface
- MTCNN - Multi-task Cascaded Convolutional Networks
- ROC-AUC - Receiver Operating Characteristic - Area Under Curve
- DFDC - Deepfake Detection Challenge

end of the report